# Summary

## Reflection

I have already worked with TCP quite some much before this module. And I had also previously rad the Smart-TCP paper (along with a lot of extra learning), so, I was already familiar with a fair amount of terminology and how TCP works. I have also done a lot of sockets programming before, both when it comes to small malware development (in go and python) and web development (javascript). However it was still nice to formally go through these concepts “formally” and learn things like sequence and acknowledgment numbers, ARQ, retransmissions, flow control, congestion control and the connection establishment/termination process. It helped connect things I had already seen and worked with before into a much more structured understanding of what TCP is actually doing underneath.

## Introduction

The transport layer sits between the application and the network layer. It’s main job is to provide logical communication between application processes running on these different hosts. The network layer is mainly responsible for getting packets from one host to another while the transport layer here extends its functions by dealing with the processes running on these hosts. This is done using TCP and UDP, along with sockets and port numbers. Port numbers are important because a host can have multiple applications communicating over the network at the same time.

So, the transport layer needs some way to know which application should receive each piece of data. This is handled with multiplexing and demultiplexing. Multiplexing is when the data from different application processes are collected and passed down to the network layer and demultiplexing happens at the receiver when the transport layer looks at the header information and sends the receives data to the correct application process.

## Sockets

A socket acts as the interface between between an application process and the transport layer. (All) applications send and recieve data over sockets and the transport protocol handles moving that data between the who hosts. With UDP, demultiplexing mainly depends on the destination port number. With TCP, a connection is identified using the combination of src IP, src port, dst IP, and dst port. Client applications usualkly use an ephemeral port, which is a temporary port that is automatically selected by the OS while servers normally listen on a fixed port.

## UDP

User datagram protocol is a connectionless transport layer protocol. There’s no handshake or connection establishment before the data is sent unlike TCP and each UDP datagram is treated independently. UDP also does not guarantee delivery and it doesn’t over retransmission if something gets lost. This makes UDP lightweight compared to TCP because of its low protocol overhead. This is why it makes more sense to use UDP for things like DNS, real time communication, and anywhere else where the speed and low latency matters a lot more than receiving every single lost packet. TCP is more suitable when the data actually needs to arrive reliably and in the correct order.

The UDP header itself is very small and it has only 4 fields: src port, dst port, length and the checksum. Each field is 16 bits (or 2 bytes), making the entire UDP header 8 bytes in size. The src port identifies the sending process, while the dst port identifies the process or the service that should recieve the datagram. The length field gives the total size of the UDP header and the UDP payload added together and the checksum is used for error detection. UDP is identified by the protocol number 17 in the IPv4 header.

The UDP checksum uses 16 bit ones complement arithmetic. The 16 bit words are added together, and if the addition results in a carry from the most significant bit (MSB), the carry is wrapped around and added back to the result. The final sum is then one's complemented by flipping all of the bits to create the checksum. At the receivers end, the received words and the checksum are added together again. If the result after wraparound is all 1s, that means no error was detected. However, the checksum only provides relatively weak error detection. If errors change the two words where one change effectively cancels out the other, the resulting sum can remain the same and corruption might not get detected.

## TCP

TCP (transmission control protocol) on the other hand is different because its a connection oriented transport layer protocol. Before application data is transferred, both sides establish a TCP connection. TCP provides point to point, full duplex communication – meaning both the sides can both send and recieve data through the same connection. Data coming from an application is first stored in a TCP send buffer, and TCP then takes chunks of the byte stream from this buffer to create TCP segments. At the receiver, TCP extracts the data and places it into the recieve buffer, where the receiving application can then read it through its socket.

The maximum amount of application data that is normally placed into one TCP segment is controlled by the MSS (Maximum Segment Size). It doesn't include the TCP or IP headers. It only refers to the application data inside the TCP segment. For example, with a 1500 byte Ethernet frame, and a normal 20 bytes IP and 20 byte TCP headers, the MSS would be 1500-40=1460 bytes.

The TCP header is also much more detailed than the UDP header. Some of the main fields include: Source Port, Destination Port, Sequence Number, Acknowledgement Number, Header Length, Flags, Receive Window, Checksum, Urgent Pointer and TCP Options. The sequence number identifies the byte stream number of the first byte of data in that segment. So, TCP sequence numbers count bytes rather than segments. The acknowledgment number represents the next byte expected from the other side. TCP uses cumulative acknowledgements – meaning an ACK for a particular byte also confirms that everything before that byte has already been received.

TCP flags controls the connection.

- SYN is mainly used establishing a connection
- ACK indicates that the acknowledgment number is valid
- FIN is used when closing a connection
- RST is to used to reset a connection
- PSH tells TCP to pass the receives buffered data up to the application rather than waiting more unnecessarily for more data.
- The Urgent Pointer works with the URG flag to identify urgent data.

TCP options can also include things like MSS, SACK permitted, timestamps and window scaling, which is why SYN and SYN/ACK segments can have larger headers than a normal ACK segment. SACK permitted tells that the host supports Selective Acknowledgment (SACK), allowing it to identify which TCP data blocks have been received.

A TCP connection is established with a three way handshake. The client first sends a SYN, the server responds with SYN, ACK and the client sends the final ACK. The sequence and acknowledgement numbers are also synchronized during this process. Note that a SYN consumes one sequence number – and this is why the acknowledgment generally increases the SYN sequence number by one. Once the final ACK is recieved, the connection is established and application data can start being transferred.

TCP can later do a graceful connection release using FIN, ACK, FIN, ACK. This lets each side independently indicate that it has finished sending data and make sure the connection is properly closed instead of just disappearing.

TCP is designed to provide reliable communication, so it needs some way to recover when data is lost. This is where the retransmissions and ARQ (Automatic Repeat reQuest) concepts come in. ARQ combines both error detection, receiver feedback and retransmission. A basic example is Stop and Wait ARQ – this is when the sender sends one packet and waits for an acknowledgement before sending the next one. If the ACK does not arrive before a timeout, the packet is retransmitted. This works but it's not very efficient on a link with longer delay because the sender will keep on spending a lot of time doing nothing while waiting for the ACK.

Two more efficient ARQ approaches are Go Back N and Selective Repeat. Both of these allows multiple packets to be in flight using a sliding window.

- With Go Back N, if one packet is lost, the sender can retransmit that packet and the following unacknowledged packets in the window. The reciever can discard out of order packets, which keeps the implementation on the receivers side simpler.
- With Selective Repeat, correctly received out of order packets are buffered, so, only the actual missing packet needs to be retransmitted. This makes it so that it will use less bandwidth, especially with larger windows, although it needs more buffering and tracking at the receiver.

Also, TCP uses timers to decide when the data should be retransmitted. The timeout should not be too short, because that would cause unnecessary retransmissions that wasn't required in the first place (due to packets arriving at different delays due to how packet switching works). However, it should not be too long because recovery from actual packet loss would take longer. TCP estimates the RTT (round trip time) using SampleRTT, EstimatedRTT and DevRTT, and then calculates a TimeoutInterval from them. Other than timeout based retransmission, TCP can also do fast transmit. If the sender receives three duplicate ACKs for the same data, it assumes that a segment is probably missing and retransmits the unacknowledged segment with the smallest sequence number without waiting for the retransmission timer to expire.

Another major feature that TCP has is flow control. The receiver has a limited receive buffer, and if the sending side sends data faster than the receiving application can read it, that buffer might eventually overflow. TCP prevents this using the recieve window (rwnd), which is advertised inside the TCP header. This tells the sender how many additional bytes the receiver currently has room for. Then, TCP uses a variable sized sliding window, allowing multiple bytes or segments to remain in flight without being acknowledged. As the ACKs arrive, the window moves forward allowing the sender to send more data. If the recieve buffer gets filled completely, the receiver can advertise a window size of zero and temporarily stop the sender.

Make sure you don't confuse flow control with congestion control. Flow control protects the receiver while congestion control is mainly about preventing too much traffic from being pushed into the network. TCP maintains another variable which is called the congestion window (cwnd). The amount of unacknowledged data the sender is allowed to have in flight is restricted by the minimum of the congestion window and recieve window: min(cwnd, rwnd).

Usually, TCP interprets packet loss or duplicate acknowledgements as a sign of network congestion. During Slow Start, the congestion window begins relatively small and grows exponentially as ACKs arrive. Once it reaches a threshold, TCP moves into Congestion Avoidance – this is where the window increases much more slowly, generally by around one MSS per RTT. Some TCP congestion control implementations also include a Fast Recovery phase after packet loss.

## Ending

Overall, the transport layer is mainly about getting data from the correct application process in one host to the correct application process in another host. UDP keeps this process simple and lightweight while TCP adds connection management, sequencing, acknowledgements, retransmissions, flow control and congestion control to provide